

Tabla por campo sensible — MTG Companion

#	Campo	Técnica elegida	Justificación técnica	Algoritmo / librería	Ubicación donde se protege	Gestión de llaves
1	User.passwordHash (contraseña)	Hashing	Solo se compara en el login, nunca se recupera. Es de baja entropía → hash lento con sal contra fuerza bruta.	bcrypt, costo 12 — bcryptjs	Backend, UsersService.register() (users.service.ts:66) y validateCredentials() (:102), antes de persistir	Sin llave. Sal aleatoria por usuario, embebida en el hash; costo (12) ajustable.
2	RefreshToken.tokenHash	Hashing	Solo se verifica que exista. Es aleatorio de 512 bits y de un solo uso → basta un hash rápido.	SHA-256 — node:crypto	Backend, AuthService.hashToken() (auth.service.ts:171)	Sin llave; el token ya es aleatorio y de un solo uso.
3	UserSettings.twoFactorSecret (secreto TOTP)	Cifrado simétrico	TOTP necesita el secreto original para generar códigos → no sirve hash. Hoy está en texto plano.	AES-256-GCM — node:crypto	Backend, capa de infraestructura: PrismaUserRepository cifra al escribir (prisma-user.repository.ts:45) y descifra al leer	FIELD_ENCRYPTION_KEY (32 bytes) en AWS Secrets Manager (prod) / .env gitignoreado (dev). Nunca en la BD ni en el código.
4	UserSettings.lastLat / lastLng (geolocalización)	Cifrado simétrico	Ubicación = dato personal (Ley 21.719). La distancia se calcula en Node → se puede descifrar ahí.	AES-256-GCM — node:crypto	Backend, mismo repositorio; columna migra de Float a String (texto cifrado)	Misma llave que #3.
5	Número de tarjeta / datos de pago	Tokenización	El sistema nunca ve la tarjeta: MercadoPago es la bóveda y paymentRef el token .	MercadoPago Checkout Pro — SDK oficial mercadopago	Proveedor externo; el backend solo persiste paymentRef (mercadopago-payment.provider.ts:55)	MERCADOPAGO_ACCESS_TOKEN en Secrets Manager; nunca llega al cliente.
6	Access token de sesión (JWT)	Firma	Importa la integridad , no el secreto: nadie debe fabricar un token. El backend verifica la firma.	HS256 (HMAC-SHA256) — @nestjs/jwt	Backend, AuthService.signAccessToken() / JwtStrategy	JWT_SECRET en Secrets Manager. Riesgo: valor por defecto 'change-me-dev-only' si falta la variable.
7	Token del QR de compraventa (código de verificación)	Firma	Garantiza que el QR apunta a una publicación real. Firmado y caduca en 3 minutos.	HS256 — @nestjs/jwt, typ: 'listing'	Backend, cards/domain/listing-token.ts	Mismo JWT_SECRET (ver #6).
8	Notificación de pago confirmado (webhook)	Firma	Verifica que la notificación viene de MercadoPago; además se re-consulta el pago a la API.	HMAC-SHA256 sobre cabecera x-signature — SDK mercadopago	Backend, MercadoPagoPaymentProvider (mercadopago-payment.provider.ts:101)	MERCADOPAGO_WEBHOOK_SECRET en Secrets Manager.

#	Campo	Técnica elegida	Justificación técnica	Algoritmo / librería	Ubicación donde se protege	Gestión de llaves
9	Credenciales de login, código 2FA, secreto TOTP en el setup, tokens de sesión — en tránsito	TLS (híbrido)	Hoy viaja por HTTP plano , incluido el secreto TOTP del setup. TLS protege todo el tránsito.	TLS 1.2/1.3 — reverse proxy Caddy + certificado Let's Encrypt (hostname nip.io/sslip.io)	EC2 del backend, delante de NestJS (:443 → :3000)	Certificado y llave privada TLS gestionados y renovados automáticamente por Caddy en la instancia.
10	Conexión backend ↔ PostgreSQL	TLS	Tráfico interno entre EC2, ya restringido por security groups. Defensa en profundidad.	TLS de PostgreSQL — sslmode=require en DATABASE_URL	deploy-backend.sh / config de Postgres	Certificado del servidor PostgreSQL.
11	Fotos subidas por los usuarios (CardPhoto → S3)	Cifrado en reposo (administrado)	Las fotos deben recuperarse y son archivos grandes → cifrado simétrico en el almacenamiento.	SSE-S3 (AES-256, por defecto en S3 desde 2023)	AWS S3 (s3.tf:34), StorageService (storage.service.ts:82)	Llaves administradas por AWS (SSE-S3).
12	Base de datos completa (cuentas, colección/inventario, transacciones, montos, cupones)	Cifrado en reposo (disco)	Se consultan y ordenan en SQL → cifrar por campo rompería la app. Se cifra el disco (hoy no).	Cifrado EBS (AES-256 vía AWS KMS, llave aws/ebs)	infra/terraform/ec2_postgres.tf → root_block_device { encrypted = true }	Llave administrada por AWS KMS (aws/ebs).
13	Secretos de configuración (JWT_SECRET , tokens MercadoPago, password Postgres, MinIO)	Gestión de secretos (no se cifran en el código: se sacan del código)	Son las llaves de todo lo anterior: nunca en el código ni junto al dato.	AWS Secrets Manager; leídos al arrancar con el rol de la instancia	infra/terraform/secrets.tf , deploy-backend.sh:79	.env , terraform.tfvars y *.tfstate gitignoreados; verificado que nunca se commitearon.
14	Credenciales AWS del administrador (desktop-runner)	Cifrado simétrico con llavero del SO	Deben recuperarse para usarse. Hoy están en texto plano fuera del repo.	Electron safeStorage (usa libsecret / Keychain / DPAPI) o, mínimo, permisos 0600	desktop-runner/apps/server/src/scripts /settings.ts:101	La llave la custodia el llavero del sistema operativo, no la app.
15	Sesión en el cliente (access + refresh token)	Almacenamiento seguro del dispositivo	En localStorage cualquier XSS puede leerlos. En Android conviene el almacén del hardware.	Android Keystore vía plugin de almacenamiento seguro de Capacitor	trading-app-ionic/src/lib/auth/AuthContext .tsx	Llave del Android Keystore, no exportable.